Institut National des Postes et Télécommunications (INPT), Rabat
Filière Smart ICT – 3e année

IA agentique pour le RIC en O-RAN

Rapport d'avancement – phases 1 à 3

Réalisé par :
Souhail Bourhim
Ghita Boulmani

Encadré par :
Pr. Mustapha Benjillali

Novembre 2026

Résumé. Ce rapport présente l'avancement du projet par rapport à la proposition d'implémentation d'octobre 2026. Il couvre les trois premières phases du plan de travail : les fondations (simulateur, plateforme OAI + FlexRIC, supervision KPM), le contrôle (politique RL, contrôle des PRB par E2) et l'agent (service LangGraph, vérificateur déterministe, console de confirmation). L'interface opérateur (Hermes Agent et la passerelle à trois outils) et l'évaluation comparative des contrôleurs feront l'objet du prochain rapport.

En résumé, la chaîne « intention → service LangGraph → vérificateur → politique A1 → xApp RL → répartition des PRB » fonctionne de bout en bout, sur le simulateur et sur une vraie cellule OAI pilotée par E2. La politique RL est conditionnée par l'objectif et passe un test conçu pour pouvoir échouer. Pour y arriver sur la plateforme, nous avons dû corriger le gNB OAI à deux endroits : un bug des mesures KPM par slice, et l'absence de contrôle des PRB par E2 dans la version utilisée. Nous indiquons aussi ce qui ne marche pas encore, en particulier une oscillation de la politique sur la plateforme réelle (section 7).

1. Rappel de l'objectif et état d'avancement

La question du projet reste la même : *comment permettre à un opérateur de piloter le RAN en langage naturel, tout en gardant des décisions rapides et en empêchant l'IA de prendre des actions dangereuses ?* La réponse proposée sépare les rôles : un LLM traduit l'intention en objectif, un vérificateur déterministe contrôle cet objectif, et une politique RL décide seule de la répartition des PRB, sans jamais attendre le LLM.

La figure 1 reprend le plan de travail de la proposition. Les phases 1 à 3 sont terminées, avec leurs points de validation ; le dernier (« intention satisfaite de bout en bout, sans Hermes ») a été atteint sur la plateforme réelle (section 6.4).

Le travail a été découpé en étapes courtes, chacune avec un critère de réussite fixé avant de commencer, des tests automatiques et une note technique. Le tableau 1 liste les étapes couvertes ici.

Étape	Contenu	Phase
M01	Simulateur gym d'une cellule à trois slices ; schéma de l'objectif RL	1
M06a	Première mise en route d'OAI + FlexRIC, capacité mesurée	1
M06	Trois slices de bout en bout sur la plateforme (cœur, gNB, un UE par slice)	1
M07a	xApp de supervision KPM ; correctif d'un bug du gNB	1
M02, M02b	Politique RL conditionnée par l'objectif, test fondé sur un oracle	2
M02c	Suivi des entraînements (MLflow), registre de modèles, intégration continue	2
M07b	Contrôle des PRB par E2 ; xApp RL sur la plateforme réelle	2, 3
M03	Vérificateur déterministe	3
M04a	Intégration : `Backend`, base partagée, politique A1, xApp RL, console	3
M04b	Service LangGraph avec un LLM local	3

Table 1 – Étapes réalisées (M06a désigne la courte étude préalable de la plateforme).

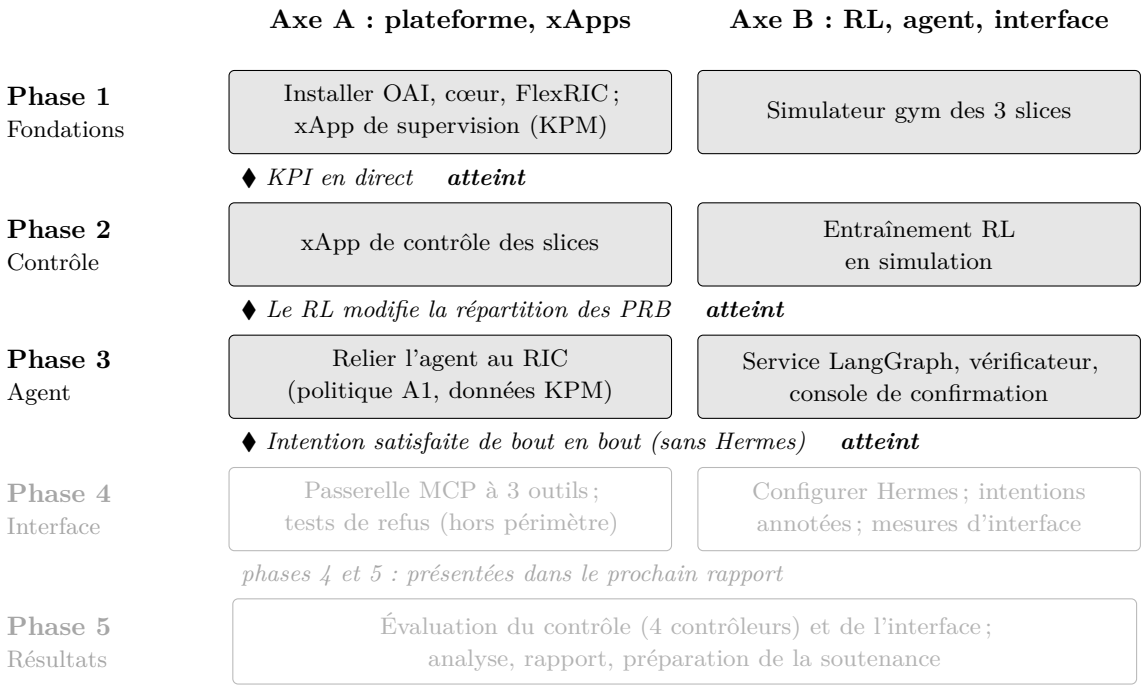


Figure 1 – Plan de travail de la proposition. En gris foncé, les phases couvertes par ce rapport.

2. Architecture réalisée

La figure 2 reprend l'architecture de la proposition. Tous les blocs du RIC et du RAN existent. La zone de l'interface opérateur est en gris : elle relève de la phase 4. Pendant les phases 1 à 3, les intentions sont envoyées directement à l'API HTTP du service LangGraph.

Trois choix structurent le code :

- **Une seule interface pour deux RAN.** La xApp RL appelle les mêmes cinq méthodes (lire les KPI, appliquer une répartition, etc.) sur le simulateur et sur la plateforme. Le service LangGraph, le vérificateur et la politique n'ont pas changé d'une ligne pour passer à OAI.
- **Une politique A1 émulée.** FlexRIC est un Near-RT RIC : il n'y a ni Non-RT RIC ni

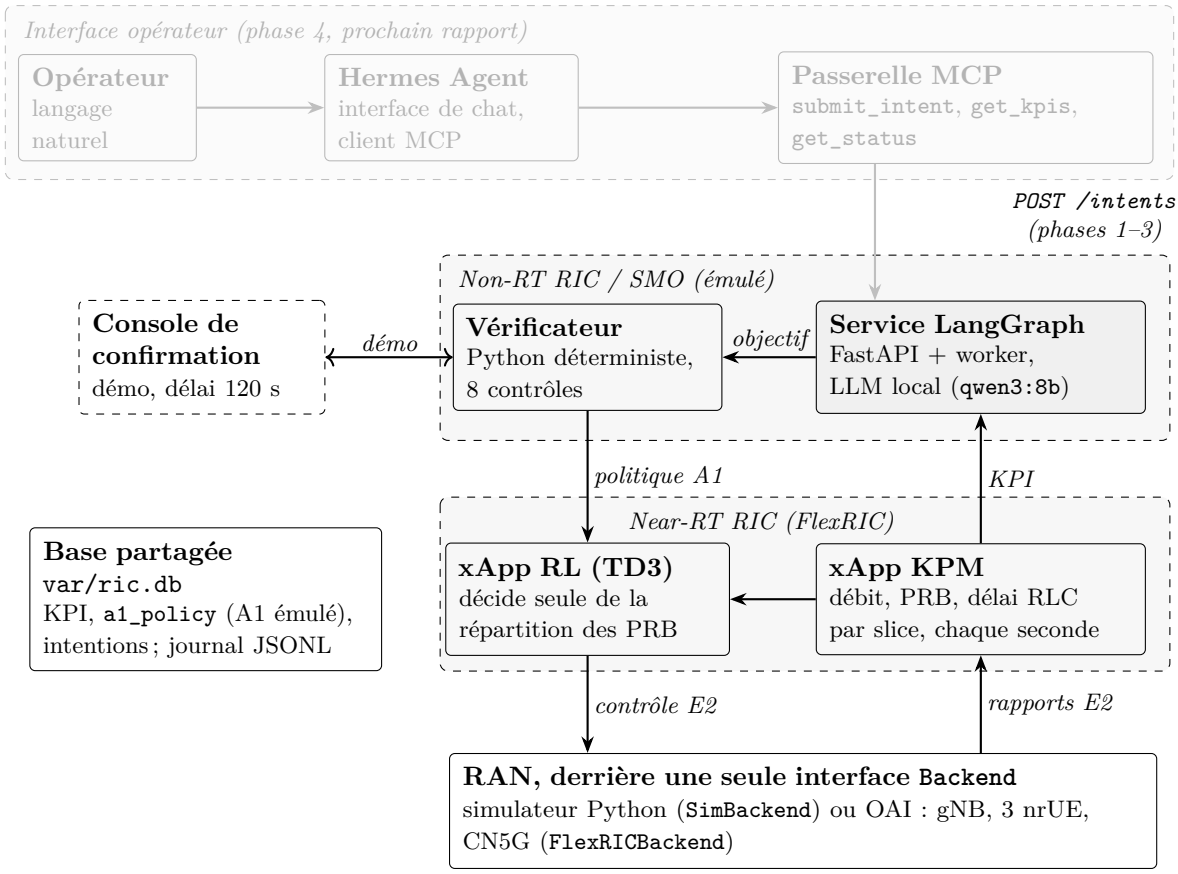


Figure 2 – Architecture réalisée. En gris : l'interface opérateur, hors du périmètre de ce rapport. Les processus ne communiquent que par la base partagée ; la xApp RL est le seul appelant de `apply_prb_split`.

interface A1 dans notre pile. L'objectif validé est donc écrit dans une table `a1_policy` sous forme d'un calendrier (segments de temps), que la xApp RL lit à chaque seconde. C'est une émulation, et nous la nommons ainsi.

- **Des invariants vérifiés par des tests.** Par exemple : le vérificateur ne fait ni appel réseau, ni tirage aléatoire, ni appel à un LLM ; la xApp RL n'importe rien du service LangGraph et n'attend jamais un LLM ; seule la xApp RL fixe la répartition des PRB.

3. Simulateur et politique RL conditionnée par l'objectif

3.1 Simulateur

Le simulateur (environnement gymnasium) représente une cellule de 106 PRB (40 MHz, espacement 30 kHz) partagée entre eMBB, URLLC et best effort, avec un pas d'une seconde. Sa capacité, 75 Mb/s, a été mesurée sur la plateforme OAI. Chaque slice est une file fluide : le trafic arrive, la part de PRB fixe le débit de service, le reste attend dans un tampon. La latence suit un modèle M/M/1 plus le temps de vidage du tampon, de sorte qu'elle monte avant la saturation et donne un signal à la politique. Le trafic suit des profils (stable, pic, rampe) avec du bruit.

L'objectif RL est un schéma pydantic : un poids par slice, un débit minimal par slice, une borne de latence pour URLLC, et une fenêtre de validité. Il fait partie de l'observation : c'est ce qui rend la politique *conditionnée par l'objectif*. Le LLM peut changer l'objectif en cours d'exécution sans ré-entraînement.

3.2 Entraînement : un premier échec, puis un test valide

Nous avons d'abord entraîné DDPG, puis TD3 (une variante plus stable de DDPG, prévue comme solution de repli ; mêmes hyperparamètres dans stable-baselines3). Toutes les politiques battaient la répartition égale, mais **le critère de conditionnement fixé à l'avance a échoué** : pour au moins une graine, la répartition changeait à peine entre deux objectifs. Nous avons gardé cet échec dans le dossier et cherché les causes, chacune mesurée avant d'être corrigée (tableau 2).

Cause trouvée	Correction
Le test lui-même ne séparait pas une politique parfaite d'une mauvaise (cas d'égalité entre slices)	Nouveau test fondé sur la récompense et sur un oracle
Déficit réel sous contraintes : sous « protéger URLLC », l'optimum gagne 0,252 par pas, la politique −0,015	Ré-étiquetage des objectifs (idée de *hindsight experience replay*) : 80 % de chaque lot est rejoué sous un autre objectif
Une « falaise » dans la récompense : au-delà de deux fois la borne, toute latence coûtait pareil	Pénalité logarithmique $\log(1 + \text{excès/borne})$
Le pic du test (×4) n'était jamais vu à l'entraînement (×1,5 à 3)	Pics jusqu'à ×5, charge ×1,2
Le filtre de lissage n'était appliqué qu'au déploiement	Filtre inclus dans l'environnement d'entraînement

Table 2 – Causes de l'échec du premier critère et corrections.

Le nouveau test repose sur une idée simple : *dire à la politique l'objectif A doit l'aider à bien faire sous A*. On compare la récompense sous A quand on lui annonce A, et quand on lui annonce un autre objectif B. Un oracle, réservé à l'évaluation, sert de référence : chaque seconde, il essaie 861 répartitions, prédit la seconde suivante avec les vraies charges futures et applique la meilleure. Un contrôleur qui ignore l'objectif échoue par construction. Les critères (avantage positif dans au moins 90 % des cas, au moins la moitié de l'avantage de l'oracle capturé, et mieux que la répartition égale) doivent être tenus *par chaque graine*.

Le protocole a été fixé avant tout entraînement : choix de la recette sur un jeu de validation, confirmation sur cinq graines nouvelles, puis test réservé, joué une seule fois. **Les cinq graines passent** (avantage positif dans 97 % des cas, 70 à 88 % de l'avantage de l'oracle, récompense 0,279 à 0,286 par pas contre 0,183 pour la répartition égale). La figure 3 montre l'effet : la même politique, sur le même pic URLLC, agit différemment selon l'objectif annoncé.

Une seconde série d'entraînement a ensuite ajouté un plancher de 5 % de PRB par slice, pour que best effort ne soit jamais affamé ; elle passe aussi le test réservé sur chaque graine. La politique déployée (« v7 ») vient de cette série.

3.3 Suivi et déploiement des modèles

Chaque entraînement et chaque évaluation sont enregistrés dans MLflow (local), avec les paramètres, le commit git et les courbes. Le modèle déployé est désigné dans un registre par l'alias `champion`, que seule une règle déterministe peut déplacer : test réservé réussi, recette réussie sur chaque graine, puis meilleur score de validation. La xApp RL charge ce modèle une seule fois au démarrage et n'appelle jamais MLflow dans la boucle de contrôle. Les tests tournent

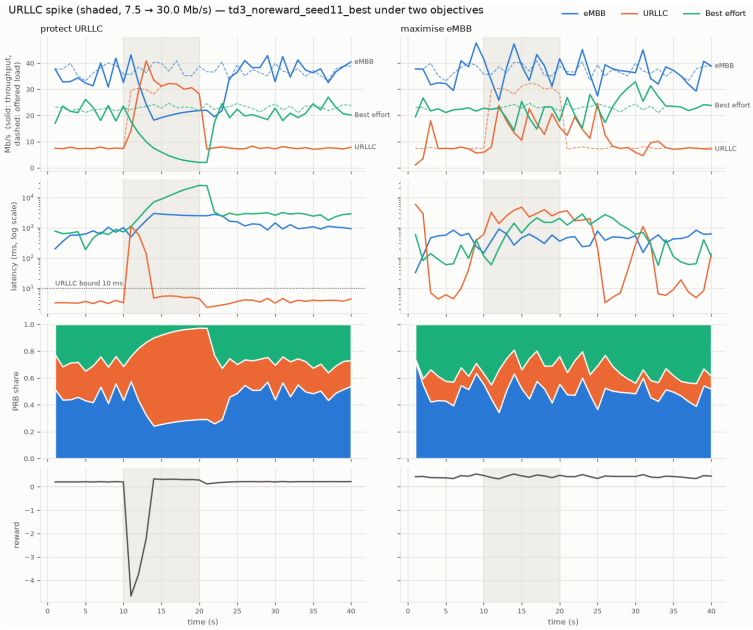


Figure 3 – Une politique validée, même pic URLLC (zone grise), deux objectifs. À gauche, « protéger URLLC » : URLLC prend l'essentiel des PRB en 3 s environ et repasse sous sa borne de 10 ms ; best effort paie. À droite, « maximiser eMBB » : URLLC reçoit peu. Lignes : débit, latence (échelle log), parts de PRB, récompense.

automatiquement à chaque *push* (GitHub Actions).

4. Vérificateur déterministe

Le vérificateur est la règle fixe entre « le LLM l'a proposé » et « le réseau l'applique ». Il prend l'objectif proposé et l'état courant (horloge, capacité, intentions actives), exécute huit contrôles dans un ordre fixe, et renvoie *accepté* ou *rejeté* avec une phrase qui contient les chiffres. Il est écrit en Python pur, sans LLM, sans aléa, sans réseau, sans lecture de l'horloge (elle lui est passée) : la même entrée donne toujours la même réponse, et un test le vérifie.

	Contrôle	Rejette
1	Schéma	Champ inconnu, poids hors $[0, 1]$ ou de somme différente de 1, valeur infinie ou NaN
2	Fenêtre	Intention sans fin ; durée restante inférieure à 10 s ou supérieure à un jour
3	Physique de la latence	Borne URLLC ≤ 2 ms : un slice URLLC vide prend déjà 2 ms
4	Budget	Cibles dont la somme des parts nécessaires dépasse la cellule
5	Plancher	Cibles qui laissent un slice sous 5 % des PRB
6	Domaine d'entraînement	Cible au-delà de 40 % de la capacité, borne hors de 5–50 ms (la politique n'a jamais vu ces cas)
7	Conflit	Budget ou plancher violé par la fusion avec les intentions actives ; la plus récente est rejetée
8	Amplitude du changement	Poids qui bouge de plus de 0,5 ou cible de plus de 30 % de la capacité en une fois

Table 3 – Les huit contrôles, dans leur ordre d'exécution. Le premier échec donne le motif.

Pour le budget, chaque cible est convertie en part de PRB. Pour URLLC avec une borne B,

rester sous B impose une charge $\rho \leq 1 - \text{base}/B$, donc une cible T demande $T/(C\,(1 - \text{base}/B))$ de la cellule : 30 Mb/s sous 3 ms demanderaient 120 % de la cellule, et sont refusés. Quand plusieurs intentions sont actives, elles sont fusionnées (plus haut débit minimal par slice, borne la plus stricte, poids de la plus récente), et c'est la *fusion* qui est vérifiée : ce qui est vérifié est exactement ce que suit la xApp RL. La figure 4 montre la décision pour toutes les paires de cibles.

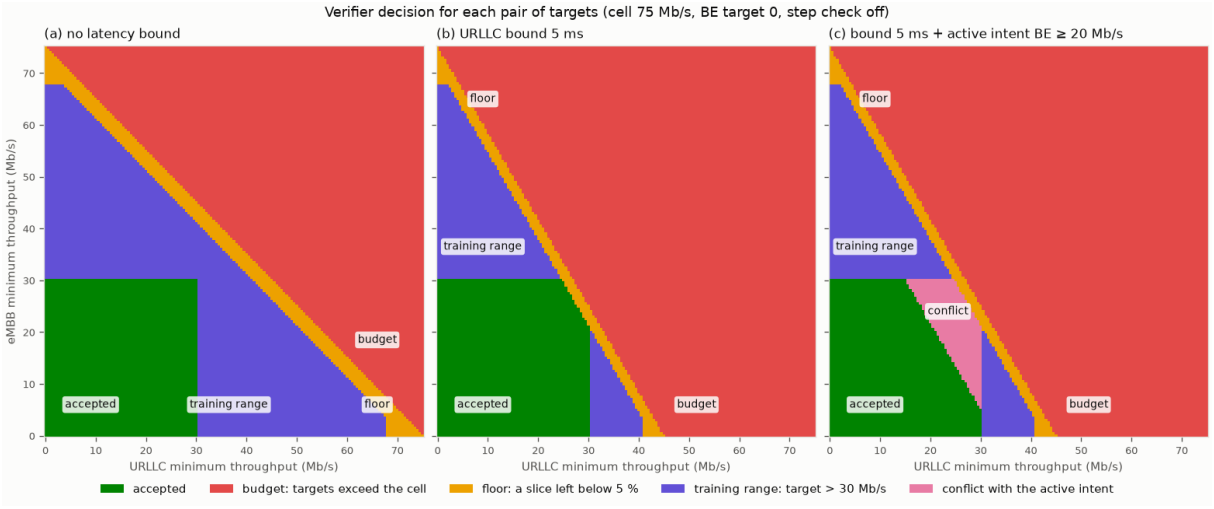


Figure 4 – Décision du vérificateur pour chaque paire de cibles (URLLC en abscisse, eMBB en ordonnée). (a) sans borne de latence ; (b) avec une borne URLLC de 5 ms, qui réduit la zone acceptée ; (c) avec en plus une intention active « best effort ≥ 20 Mb/s », qui crée une zone de conflit. Les couleurs indiquent le contrôle qui rejette.

Les tests couvrent 13 cas acceptés et 24 cas rejetés, chacun avec le contrôle et le motif attendus, y compris les cas limites (cibles exactement au budget, fenêtres qui se touchent, arrondis flottants).

5. Service LangGraph et intégration

5.1 Processus et base partagée

Les composants tournent comme des processus séparés qui ne communiquent que par une base SQLite (`var/ric.db`) et un journal d'événements : la xApp RL, le service LangGraph (API FastAPI et un *worker*), la console de confirmation, et un exportateur Prometheus pour un tableau de bord Grafana. `POST /intents` ne fait qu'enregistrer la demande ; le *worker* la traite ensuite. Chaque seconde, la xApp RL lit le segment de politique A1 en vigueur, construit l'observation (avec la même fonction qu'à l'entraînement), applique la répartition et écrit les KPI. Si la base est occupée, elle garde le dernier segment et ne l'attend pas. Une intention qui expire n'a besoin d'aucun processus : l'horloge passe simplement au segment suivant du calendrier.

5.2 Boucle de l'agent

La boucle prévue dans la proposition a été réalisée sous forme d'un graphe LangGraph explicite (figure 5). Le LLM extrait d'abord les *faits* de la phrase (slice prioritaire, débits, borne, durée), puis choisit un outil à la fois. Les chiffres de conformité sont calculés par du code ; le LLM ne fait que décider « terminé » ou « réviser ».

Quelques choix de conception :

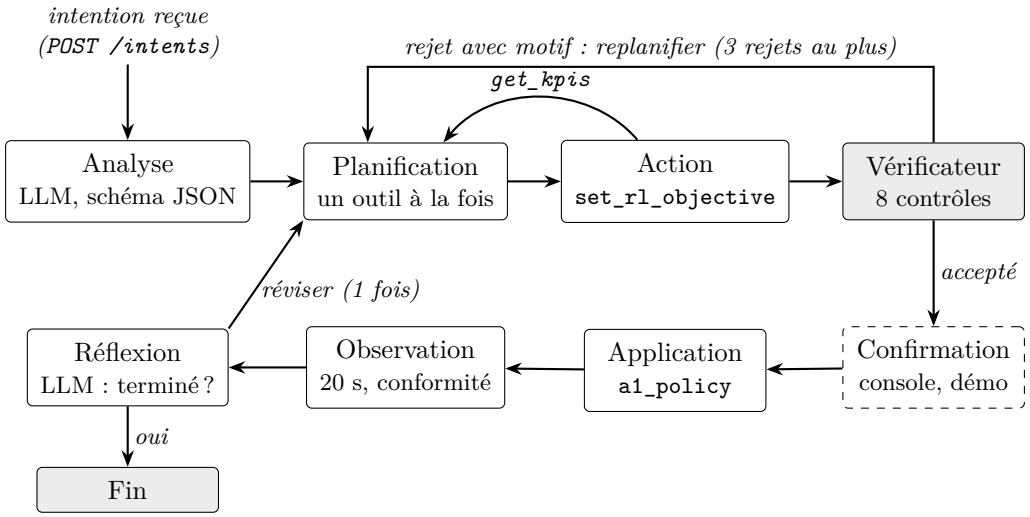


Figure 5 – Boucle du service LangGraph, telle qu'elle est implémentée. En évaluation automatique, l'étape de confirmation est sautée.

- **Faits et décisions séparés.** L'analyse ne fait qu'extraire ce qui a été dit ; la planification décide. L'extraction peut ainsi être évaluée seule contre des annotations.
- **Une importance de 0 à 10 par slice plutôt que des poids.** Un modèle de 8 milliards de paramètres gère mal trois décimales de somme 1 ; la normalisation est faite par du code, et le vérificateur contrôle le résultat.
- **En démonstration, les chiffres de l'opérateur ne sont pas modifiés.** Si le LLM propose une cible différente de celle demandée, la proposition est renvoyée ; si le vérificateur rejette les valeurs demandées, l'intention se termine avec son motif.
- **La confirmation se fait dans une console séparée**, avec un délai d'expiration de 120 s, et non par un outil du LLM, qui pourrait sinon se l'accorder lui-même.

5.3 Choix du modèle

Le LLM tourne en local avec Ollama. La règle de choix a été écrite avant les essais : le modèle doit tenir entièrement dans la mémoire du GPU (8 Go), puis il est classé par le nombre d'intentions entièrement bien lues sur 20 intentions annotées de réglage, et départagé par la latence.

Modèle	Intentions correctes	Latence médiane	Part sur le GPU	Décision
`qwen3:8b`	**19/20**	**1,3 s**	**100 %**	**retenu**
`qwen3.5:9b`	19/20	2,1 s	87 %	exclu (ne tient pas)
`ministral-3:8b`	19/20	1,4 s	89 %	exclu (ne tient pas)
`llama3.1:8b`	17/20	0,9 s	100 %	2 de moins
`granite4.2:8b`	6/20	0,4 s	100 %	champs numériques omis

Table 4 – Comparaison des modèles locaux sur le jeu de réglage.

Les erreurs avaient toutes la même cause (une réponse « compris » sans aucun champ rempli). Une règle ajoutée au prompt et une vérification dans le code les corrigent : 20/20 sur le jeu de réglage, un chiffre optimiste par construction puisque le prompt a été ajusté sur ces phrases. Le score sur des intentions jamais vues sera donné avec l'évaluation de l'interface.

6. Plateforme OAI + FlexRIC

6.1 Trois slices de bout en bout

La plateforme tourne dans Docker sous WSL2, sans matériel radio : cœur OAI CN5G v2.2.1, gNB OAI v3.0.0 en mode rfsim avec son agent E2, Near-RT RIC FlexRIC v3.0.0, et trois nrUE, un par slice. Les trois slices sont déclarés de façon cohérente dans toutes les couches (AMF, SMF, UPF, base des abonnés, gNB, UE) ; un test vérifie que les fichiers de configuration concordent.

Slice	S-NSSAI (SST, SD)	DNN	Sous-réseau UE	5QI
eMBB	1, 000001	`embb`	10.0.1.0/24	9
URLLC	2, 000001	`urllc`	10.0.2.0/24	7
Best effort	1, 000002	`be`	10.0.3.0/24	9

Table 5 – Configuration des slices sur la plateforme.

Mesures avec iperf3 : environ 71 Mb/s pour un slice seul. L'ensemble consomme environ 4 Go de RAM et 4 à 7 cœurs sur 32, ce qui laisse la place aux xApps, à l'agent et au LLM. Le résultat principal de cette étape : **les slices existent dans la signalisation, mais l'ordonnanceur d'OAI les ignore**. Avec 40 Mb/s offerts à chacun, chaque slice reçoit un tiers de la cellule et le délai aller-retour monte à environ 16,6 s pour tous, URLLC compris. C'est exactement le manque que la xApp RL doit combler.

6.2 Supervision KPM, et un bug du gNB

La xApp de supervision s'abonne au modèle de service KPM avec un abonnement par slice (filtré sur le S-NSSAI complet) et trois mesures : débit, PRB utilisés, délai RLC. Une partie en C sur le RIC imprime les rapports ; une partie en Python les transforme en enregistrements KPI, au même format que ceux du simulateur. Le service LangGraph et Grafana lisent ainsi les données réelles sans aucune modification.

Avec un abonnement par slice, OAI v3.0.0 rendait des valeurs absurdes (figure 6, à gauche). La cause : le gNB calcule chaque débit comme une différence de compteurs, et rangeait la « valeur précédente » par position dans la liste d'un abonnement. Avec un UE par slice, les trois abonnements partageaient la même case, et chaque différence était prise contre le compteur d'un autre slice. Un correctif d'environ 50 lignes garde la valeur précédente par UE ; nous construisons notre propre image du gNB. Après correction, KPM suit iperf3 avec une erreur médiane de 2,5 %.

Deux remarques sur les mesures : le « délai » mesuré par KPM est le temps d'attente dans la file RLC du gNB, pas une latence de bout en bout ; et une cellule pleine affiche 74 % de PRB utilisés, car la mesure inclut les créneaux montants du motif TDD. Nous corrigeons ce facteur (0,743), qui se déduit de la configuration du gNB.

6.3 Contrôle des PRB par E2

L'étude préalable avait montré que **le contrôle des PRB par slice n'existe pas dans OAI v3.0.0** : le modèle de service SLICE de FlexRIC y est une coquille vide. Nous avons donc modifié le gNB :

- un ordonnanceur qui garde chaque slice dans sa propre plage de PRB, en réutilisant l'ordonnanceur *proportional fair* d'OAI à l'intérieur de chaque plage ;

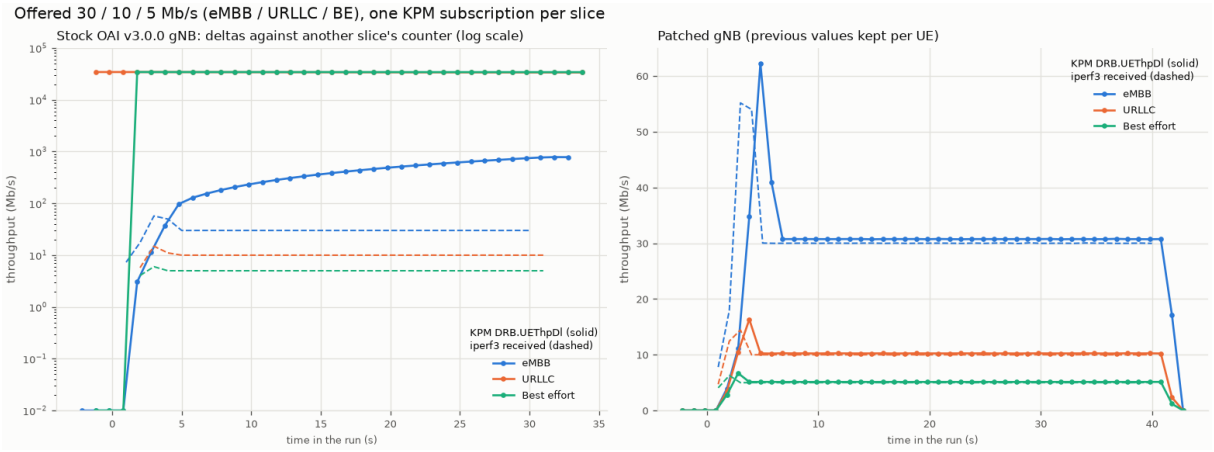


Figure 6 – Débit KPM par slice (trait plein) et débit reçu par iperf3 (pointillés), pour 30 / 10 / 5 Mb/s offerts. À gauche, gNB d'origine (échelle log) : les valeurs n'ont pas de sens. À droite, gNB corrigé : KPM et iperf3 se superposent.

- un vrai traitement du message de contrôle SLICE, qui refuse les plages hors cellule, qui se chevauchent ou de moins de 5 PRB, et renvoie la raison dans l'acquittement.

Une petite xApp en C transforme « PRB par slice » en messages de contrôle E2, et `FlexRICBackend` place la plateforme derrière la même interface que le simulateur. Le gNB répond en 0,25 ms. Le débit suit les PRB de façon linéaire : 0,654 Mb/s par PRB, de 5 à 96 PRB (figure 7), soit environ 8 % de moins que l'hypothèse du simulateur.

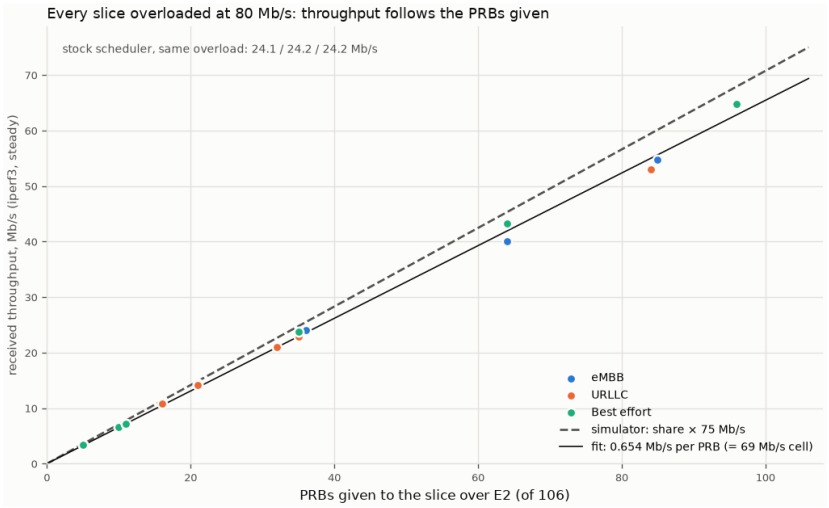


Figure 7 – Débit reçu en fonction du nombre de PRB attribués par E2, chaque slice étant surchargé (80 Mb/s offerts). Trait plein : ajustement mesuré ; tirets : hypothèse du simulateur.

6.4 La boucle fermée sur la vraie cellule

Nous avons rejoué sur la plateforme le scénario de démonstration du simulateur : un pic URLLC de 7,5 à 30 Mb/s pendant 240 s, soit 90 Mb/s offerts à une cellule d'environ 71 Mb/s. Avec l'ordonnanceur d'origine, URLLC reçoit 14,7 Mb/s sur 30 ; avec la xApp RL, 28,4 Mb/s, au prix du best effort, ramené à son plancher de 5 %.

Enfin, une intention a été soumise pendant ce pic : « Garantis 20 Mb/s au slice URLLC pendant 5 minutes. » Le service LangGraph l'a analysée, le vérificateur l'a acceptée, l'opérateur l'a confirmée

dans la console, et la xApp RL a suivi le nouvel objectif dès la seconde suivante (figure 8). Une demande précédente à 25 Mb/s avait été rejetée par le vérificateur : le changement dépassait sa limite d'amplitude (30 % de la capacité).

Pendant le pic	Part URLLC	Débit URLLC	Secondes à $\geq$ 20 Mb/s	File URLLC (médiane)
Avant l'intention (36 s)	0,25	14,2 Mb/s	28 %	6,3 s
Intention en vigueur (184 s)	0,49	31,8 Mb/s	**95 %**	**0,36 s**

Table 6 – Effet de l'intention sur la vraie cellule (mesures KPM, une exécution).

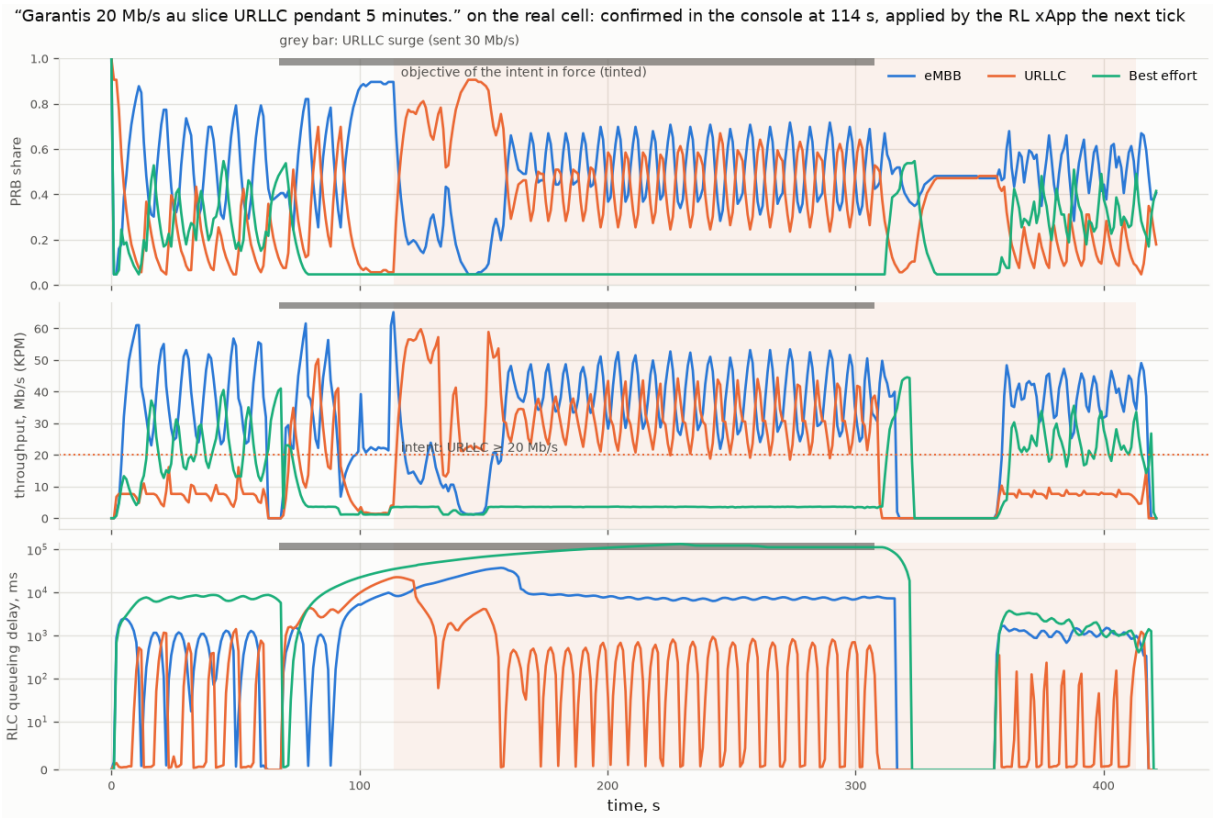


Figure 8 – L'intention appliquée sur le gNB réel pendant le pic URLLC (barre grise). Zone teintée : objectif de l'intention en vigueur. De haut en bas : parts de PRB, débit KPM (pointillé : cible de 20 Mb/s), délai de file RLC (échelle log). L'oscillation des parts est discutée en section 7.

7. Difficultés et limites

Difficulté	Ce que nous avons fait	Ce qui reste
Le premier critère de conditionnement par l'objectif a échoué	Causes mesurées (test mal conçu, quatre défauts d'entraînement), nouveau test fixé avant l'entraînement, réussi sur chaque graine	Validé en simulation uniquement
Bug KPM d'OAI v3.0.0 avec plusieurs abonnements	Correctif d'environ 50 lignes, image du gNB maison	Correctif non proposé à OAI
Pas de contrôle des PRB par E2 dans OAI v3.0.0	Ordonnanceur par slice et traitement du contrôle SLICE ajoutés au gNB	gNB à reconstruire depuis les sources ; les PRB réservés et inutilisés sont perdus (best effort seul passe de 64,7 à 19,7 Mb/s à côté d'un eMBB inactif)
FlexRIC ne transmet pas la réponse du gNB à la xApp	Le refus reste visible dans le journal du gNB et dans les captures	La xApp ne distingue pas un ordre accepté d'un ordre refusé
La politique oscille sur la plateforme (part eMBB entre 0,22 et 0,84, période 10–12 s)	Cause trouvée : la politique voit l'effet de ses décisions en retard (une seconde mesurée, deux d'après la simulation), ce qu'elle n'a jamais vu à l'entraînement. Lissage plus fort : part de la cellule déplacée par seconde de 10,8 à 4,3 %, secondes avec URLLC $\geq$ 20 Mb/s de 69 à 90 %	L'oscillation diminue sans disparaître ; le délai de file URLLC reste de l'ordre de la seconde. Un ré-entraînement avec retard est à décider
KPM ne mesure pas la charge offerte	Sur la plateforme, la xApp RL lit le débit que le script de trafic envoie ; sans cette information, elle se fige après le pic	Estimer la demande à partir de mesures du gNB (tampon RLC)
Deux exécutions de la même politique sur le même scénario se sont comportées différemment	Constaté et noté	Répéter chaque mesure (prévu en phase 5)
Plateforme simplifiée	Canal rfsim idéal, un UE par slice	Résultats à lire comme une validation de la chaîne, pas comme une mesure radio réaliste

Table 7 – Difficultés rencontrées et limites actuelles.

8. Suite du travail

Les deux phases suivantes reprennent le plan de la proposition et seront présentées dans le prochain rapport :

- **Phase 4, interface :** passerelle MCP qui expose exactement `submit_intent`, `get_kpis` et `get_status` ; Hermes Agent isolé dans un conteneur, outils généralistes désactivés ; tests de demandes hors périmètre.
- **Phase 5, résultats :** comparaison des quatre contrôleurs (répartition statique, RL seul, LLM seul, LLM + RL) sur les mêmes scénarios, en simulation et sur la plateforme ; évaluation de l'interface (extraction sur des intentions annotées jamais vues, appels d'outils, tentatives hors périmètre).

Les limites de la section 7, en particulier l'oscillation et la charge offerte, seront traitées en parallèle.

9. Points à discuter avec l'encadrant

- Les deux modifications du gNB OAI (correctif KPM, contrôle des PRB par slice) peuvent-elles être présentées comme une contribution du projet ? Faut-il les proposer au projet OAI ?
- Pour l'oscillation sur la plateforme : ré-entraîner la politique avec un retard de mesure, ou garder le lissage plus fort et le documenter comme limite ?
- La lecture de la charge offerte depuis le générateur de trafic est-elle acceptable pour la suite, ou faut-il d'abord la remplacer par une estimation mesurée ?
- Quelle forme et quelle date pour le prochain point d'avancement ?

Références

[1] Rezazadeh et al., apprentissage par renforcement profond multi-agents pour le slicing du RAN.

[2] Wu et al., IA agentique pilotée par LLM pour la résilience en O-RAN.

[3] Bao et al., LLM-hRIC : contrôle hiérarchique LLM + RL dans le RIC.

[4] Fujimoto et al., *Addressing Function Approximation Error in Actor-Critic Methods* (TD3), ICML 2018.

[5] Andrychowicz et al., *Hindsight Experience Replay*, NeurIPS 2017.

[6] OpenAirInterface : gNB et nrUE v3.0.0, OAI CN5G v2.2.1 (`https://openairinterface.org`).

[7] FlexRIC v3.0.0 (`https://gitlab.eurecom.fr/mosaic5g/flexric`).

[8] O-RAN Alliance, E2SM-KPM v3.00 ; 3GPP TS 28.552 (mesures de performance).

[9] LangGraph, Ollama, stable-baselines3, gymnasium, MLflow.